Flight Delay Prediction at 05:00: Gradient Boosting Against Strong Baselines on NYC 2013 Flights

Rishik Chowdary Karuturi, Nakkina Lakshmi Praneeth, Balanagu Sesha Srikar,
Shaik Junaid Sami, Siraparapu Shiva Sankar Mani Kanta
Team T107, Section D, Advanced Machine Learning (Phase 1)
[Institute] Guide: [faculty name]

Abstract—We predict, at 05:00 on the day of a flight, whether a New York departure will arrive at least 15 minutes late, using only the published schedule, static aircraft data and the delays of earlier days. On 336,776 flights from 2013 we follow the full machine-learning life cycle: problem definition, data collection, EDA, preprocessing, a chronological split, leakage-safe feature engineering and selection, baselines, model selection, honest tuning of gradient boosting with rolling months, and a single evaluation on November–December (base rate 0.258). Tuned XGBoost (test PR AUC 0.390), logistic regression (0.403) and a hybrid that boosts from the logistic-regression prediction (0.403) all beat a majority-class (0.258) and a rule-of-thumb baseline (0.369), but their 95% confidence intervals overlap: at this decision time gradient boosting does not significantly beat a regularised linear model. We show why: early stopping on the scored month inflates boosting's validation score, the useful tree count is unstable between months, and tree splits on absolute delay-rate levels suffer from seasonal drift. An appendix records every design decision, the evidence for it and what happened.

I. Introduction

Late arrivals cascade through crews, gates and connections. A risk score available before the first departure lets planners add buffer or warn passengers. Following the course's five-step blueprint, the goal is to rank flights by delay risk (PR AUC) and to produce a flag list with precision ≥ 0.5. The problem is binary classification: the U.S. BTS defines a flight as delayed when it arrives 15 or more minutes late, and delay minutes are heavily skewed (skewness 3.36). The project brief asks for gradient boosting; we test whether it is justified against simple baselines, as Phase 7 of the life cycle requires. The work is split across five notebooks, one per team member, each writing its own module in src/fdp/ and saving its results to reports/, from which this report is generated.

II. Related Work

[Each member adds 1–2 papers they have read, e.g. Rebollo & Balakrishnan (2014) on network delay state; Sternberg et al. (2017) review; Chen & Guestrin (2016) XGBoost; Saito & Rehmsmeier (2015) PR vs ROC.]

III. Problem, Data and Split (Phases 1, 2, 5)

Decision time. Inputs must be known at 05:00 on day D: the published schedule for D, static aircraft facts and outcomes of earlier days. We first considered "end of the previous day", but about 3% of the previous day's flights land after midnight, so 05:00 is when that history is really complete. Departure and arrival times, delays and air time are never inputs.

TABLE I
Chronological split (labelled flights).

Block	Dates	Flights	Share	Delay rate
train	01-15 – 08-31	205,642	65.2%	0.268
valid	09-01 – 09-30	27,010	8.6%	0.137
calib	10-01 – 10-31	28,618	9.1%	0.176
test	11-01 – 12-31	53,991	17.1%	0.258

Data. The Kaggle Flights (nycflights13) tables contain 336,776 scheduled departures from JFK, LGA and EWR plus a planes table. 8,255 cancelled and 1,175 diverted flights have no arrival delay, so their label is undefined; they are kept on the schedule (they count towards congestion) but never labelled, leaving 327,346 labelled flights, 0.245 of them delayed.

Metric first. The metric was fixed before any model: PR AUC (average precision), with ROC AUC, Brier score, expected calibration error (ECE), lift = PR AUC / base rate, and 95% confidence intervals from a day-block bootstrap (flights on one day share weather, so days are resampled). Accuracy is rejected: predicting "always on time" scores 0.863 accuracy in September and catches no delays.

Split. The split is chronological (Table I); January 1–14 only feeds history features. The monthly delay rate moves from 0.137 (September) to 0.336 (December, Fig. 1). This base-rate shift recurs in every later result.

IV. EDA and Preprocessing (Phases 3, 4)

All EDA uses the training block only. Aircraft year is missing for 17.47% of flights, concentrated in MQ (95.1%) and AA (69.5%), whose tail numbers are absent from the planes table. Because this is systematic, deleting rows would bias two airlines; we impute the training median and add a missing-indicator column. 9.10% of arrival delays are IQR outliers; they are real and cannot distort a yes/no label, so they are kept. There are 0 duplicates. The delay rate rises from 0.105 at 05:00 to 0.395 at 20:00 (delay propagation), differs more than twofold between carriers, and is lowest on Saturdays (Fig. 2).

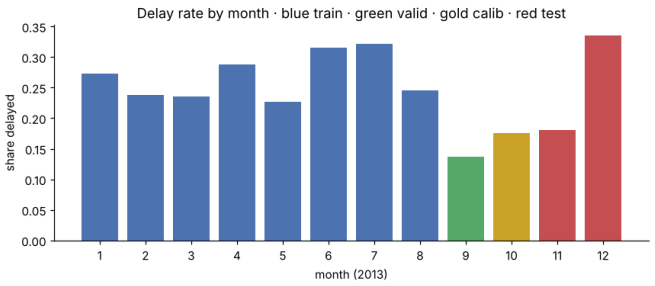


Fig. 1. Delay rate by month: train (blue), valid (green), calib (gold), test (red).

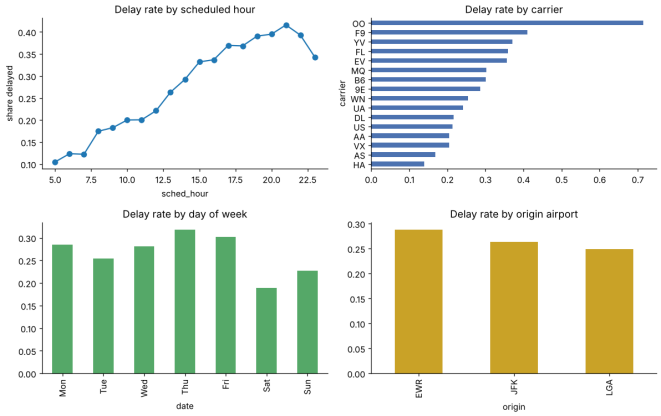


Fig. 2. Training delay rate by hour, carrier, weekday and origin.

The daily delay rate has lag-1 autocorrelation 0.343, a PACF cut-off after lag 1 (lag-2 PACF -0.109) and lag-7 autocorrelation 0.008; the ADF test rejects a unit root ($p < 10^{-10}$). Yesterday therefore carries most of the memory (a 1-day history window), and the weekly pattern is a fixed day-of-week effect (calendar features), not weekly memory. Longer 7- and 28-day windows are used only for small groups whose daily rates are noisy.

With 0.268 of training flights delayed, imbalance is moderate, so we do not re-sample (it distorts probabilities). Linear models get median imputation, scaling and one-hot encoding (levels with < 50 training flights grouped); trees get ordinal codes and no scaling; XGBoost uses native categorical splits. A standard and a robust scaler tied on September (PR AUC 0.1974 vs 0.1978); the standard scaler is kept. Every statistic is fitted inside a scikit-learn pipeline on training rows only.

V. Features and Selection (Phase 6)

We build 41 numeric features in five groups plus carrier, origin and destination: cyclic (sin/cos) schedule time, calendar (day of week, weekend, days to a federal holiday), aircraft, congestion counted from the published schedule, and trailing delay history for 12 (grouping, window) pairs. History rates are shrunk towards the overall rate, $\hat{r}_g = (k_g + \alpha\, r_0)/(n_g + \alpha)$ with $\alpha = 10$, and each comes with a log-volume feature. Windows end on day D−1 (a rolling sum followed by a one-day shift). A unit test scrambles every outcome from a cutoff date onward and checks that features on that date are unchanged; it passes. Month and day-of-year are deliberately excluded: test months never occur in training, so they would act as date IDs.

TABLE II
Feature selection (logistic regression PR AUC). None beats all features on both months.

Method	n	September	October
all features	41	0.261	0.306
filter: top 10 by MI	10	0.240	0.263
VIF pruning (<= 10)	34	0.256	0.302
wrapper: SFS (12)	12	0.279	0.281
embedded: L1, C=0.002	25	0.257	0.302
PCA (24 comps, 95% var)	24	0.268	0.292

Adding groups one at a time to logistic regression, all five together score best on both September (0.261) and October (0.306). History gives the largest September gain (+0.052) but almost none in October (+0.002): which group matters depends on the month.

We tested five selection methods and adopt one only if it beats all features on both held-out months (Table II). None does. Sequential forward selection, written from scratch, reached 0.279 on September, the month it selected on, but fell to 0.281 on October against 0.306 for all features: it over-fitted its selection month. 13 features have VIF > 10, yet pruning them lowered both months, because collinearity harms coefficient interpretation, not prediction. PCA needs 24 components for 95% variance. All features are kept.

VI. Baselines and Model Selection (Phases 7, 8A)

Baselines on September / October: majority class 0.137 / 0.176; a planner's rule, the flight number's 28-day delay rate, 0.190 / 0.269; scheduled hour 0.188 / 0.229.

Logistic regression was implemented from scratch as maximum-likelihood estimation by gradient descent, $\nabla L = X^\top(\sigma(Xw) - y)/n + \lambda w$, with batch, mini-batch (256) and stochastic updates. Batch GD reached a training log loss of 0.5279 against 0.5278 for scikit-learn. A learning rate of 0.01 was slow (0.6195 after 100 epochs) and 8 diverged (2.3942). The L2 strength chosen on September is $C = 0.001$; train and September scores hardly change with C, so the linear model is limited by bias, not variance. A single decision tree memorises: unlimited depth scores 1.000 on train and 0.140 on September.

Model families were compared on rolling months (train on all months before July, August or September; score that month) with fixed, untuned settings (Table III). Logistic regression led; untuned XGBoost was third. Since default hyper-parameters are not a fair test of boosting, it proceeds to honest tuning.

TABLE III
Model families on rolling months (PR AUC, fixed settings).

Model	Jul	Aug	Sep	Mean	Fit s
Logistic regression	0.557	0.406	0.262	0.408	3.6
Decision tree	0.486	0.353	0.145	0.328	1.9
Random forest	0.554	0.389	0.216	0.386	8.0
Gradient boosting (untuned)	0.559	0.369	0.170	0.366	2.3

TABLE IV
Same configuration, two protocols (PR AUC).

Month	Early-stop on scored month	Honest protocol
Jul	0.571	0.545
Aug	0.392	0.376
Sep	0.242	0.225
mean	0.402	0.382

VII. Gradient Boosting (Phases 8B, 8C)

Boosting fits shallow regression trees to the negative gradient of log loss, $r = y - p$, with Newton leaf values $\sum r / \sum p(1-p)$. Our from-scratch implementation gives the same September log loss (0.4165) and PR AUC (0.208) as scikit-learn.

The early-stopping trap. Early-stopping on the month that is then scored lets the stopping rule see the answers. For one fixed configuration this inflated the mean rolling PR AUC from 0.382 to 0.402 (Table IV). Our protocol for month m: train on months before $m-1$, early-stop on $m-1$, refit on all months before m with that many trees, and score m.

Tuning. A random search over 8 of 288 XGBoost configurations, each scored honestly, selected learning_rate=0.03, max_depth=3, min_child_weight=50, subsample=0.7, colsample_bytree=0.5, reg_lambda=1.0 (mean 0.386). All configurations lay within 0.010 of each other and below logistic regression; shallow trees dominated, and the tree count chosen by early stopping varied widely between folds ([40, 110, 13] for the best configuration).

Hybrid. XGBoost can start from any initial prediction: with base_margin set to logistic regression's logit, the trees learn only corrections, and zero trees means plain logistic regression. Table V compares the three candidates on the same honest folds.

Final training. The tree count is the median of the fold counts: 40 for boosting and 16 for the hybrid (folds [16, 1219, 15]). Early stopping on September alone would have kept only 1 tree. All three models are refitted on January 15 – September 30 and Platt-calibrated on October.

VIII. Results on the Test Months (Phase 9)

The test months were scored once (Table VI, Fig. 3). All learned models clearly beat both baselines, and the three learned models' 95% intervals overlap heavily: the hybrid and logistic regression are practically identical,

TABLE V
Rolling-month PR AUC of the final candidates.

Model	Jul	Aug	Sep	Mean
Logistic regression	0.556	0.406	0.266	0.409
Gradient boosting (tuned)	0.548	0.391	0.219	0.386
Hybrid: boosting from LR	0.559	0.391	0.268	0.406

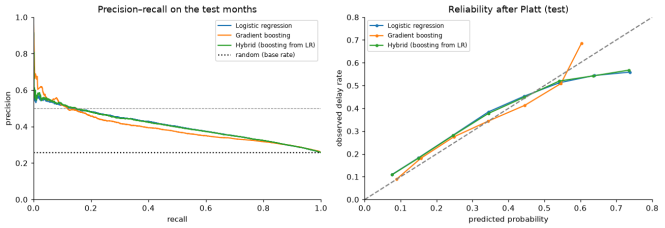


Fig. 3. Test precision–recall curves and reliability after Platt scaling.

and boosting is 0.013 lower, well inside the interval width. Platt scaling lowered boosting's calibration error but raised logistic regression's and the hybrid's: their maximum-likelihood probabilities were already calibrated, and October's lower base rate pulled them down.

Business mapping. Table VII turns scores into a flag list per 1,000 test flights. A score threshold fixed in October, rule (a), keeps precision at 0.538–0.547 but flags only 25–26 flights and catches 0.053–0.056 of delays, because the delay rate rose from 0.176 to 0.258. Flagging when the calibrated probability is ≥ 0.5, rule (b), keeps precision at 0.517–0.529 and flags 42–49 flights (recall 0.085–0.097), roughly twice as many true delays as a random list of the same size. Rule (b) is the policy we can state in advance.

IX. What Drives the Model

Importance and ablation use October, never the test set. By permutation importance boosting relies most on scheduled hour (0.01726 drop in PR AUC) and the flight number's 28-day delay rate (0.01424). Adding feature groups to boosting one at a time, October PR AUC peaks at 0.303 with schedule, calendar and aircraft, falls to 0.291 with congestion and to 0.264 with history, while logistic regression gained from both groups. Trees split on absolute levels ("28-day rate > 0.31") learned in a high-delay summer; in calmer autumn months those thresholds sit in the wrong place, whereas a linear model only rescales the rate. The same drift shows by slice: boosting's lift is 1.38 in November against 1.58 for logistic regression, and December is hard for every model (about 1.35).

X. Discussion and Threats to Validity

The decision log in the appendix lists all 35 decisions and their outcomes. Most protective decisions did what they were meant to: the chronological split and day-block intervals prevented an inflated or over-confident claim, the two-month veto stopped SFS from shrinking the feature set, and the honest protocol removed a +0.020

TABLE VI
Test results, Nov–Dec 2013 (base rate 0.258). CI: 95% day-block bootstrap. ECE: raw / after Platt.

Model	PR AUC	95% CI	ROC AUC	Lift	Brier	ECE raw / Platt
Majority class	0.258	–	0.500	1.00	0.192	0.010
Rule: flight's 28-day delay rate	0.369	–	0.638	1.43	0.184	0.036
Logistic regression	0.403	[0.351, 0.459]	0.670	1.56	0.179	0.029 / 0.035
Gradient boosting	0.390	[0.336, 0.451]	0.654	1.51	0.182	0.034 / 0.023
Hybrid (boosting from LR)	0.403	[0.352, 0.458]	0.670	1.56	0.179	0.028 / 0.032

TABLE VII
Flag list per 1,000 test flights.

Model	Rule	Prec.	Recall	Flagged	Delayed	Random
LR	(a)	0.538	0.055	26.4	14.2	6.8
LR	(b)	0.525	0.086	42.6	22.3	11.0
Boosting	(a)	0.547	0.053	25.2	13.8	6.5
Boosting	(b)	0.517	0.097	48.6	25.2	12.6
Hybrid	(a)	0.547	0.056	26.2	14.4	6.8
Hybrid	(b)	0.529	0.085	41.6	22.0	10.8

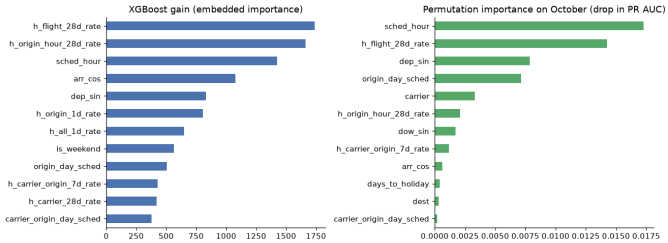


Fig. 4. XGBoost gain and permutation importance on October.

optimism from boosting's tuning score. Limitations: (i) $C = 0.001$ was chosen on September and then reused in the rolling comparison whose September fold is the same month (0.2655 vs 0.2624 with $C = 1$), and boosting's configuration was likewise picked on the folds it is reported on, so both rolling means are slightly optimistic; the test set is unaffected. (ii) The search covered 8 of 288 configurations, although their spread was small. (iii) The hybrid's August fold stopped at 1219 trees against 16 and 15 in the other folds; the median rule absorbs this, but it shows how unstable the transferable correction is. (iv) Ablation is based on one month and a fixed tree count. (v) One year of data cannot separate season from trend. (vi) Notebooks ran on different machines (notebook 04: Python 3.14.7, scikit-learn 1.9.1, XGBoost 3.4.1); re-runs may differ in the third decimal.

XI. Conclusion

At a 05:00 decision time, gradient boosting does not significantly outperform a regularised logistic regression; both clearly beat simple baselines, and a hybrid that boosts from the linear prediction matches the linear model. The useful information at 05:00 is mostly recent delay history, which a linear model uses as well as trees and transfers better across seasons. Honest tuning matters: a common early-stopping shortcut would have overstated boosting. Future work: drift-robust (relative) history features, information known at 05:00 such as weather forecasts and planned traffic programmes, more years of data, and temporal deep learning in Phase 2. [Phase 2 plan.]

Contributions

Rishik: problem definition, data, split, metrics, report. Praneeth: EDA, preprocessing. Srikar: features, selection, leakage test. Sami: baselines, gradient descent, model selection. Mani Kanta: gradient boosting, tuning, evaluation.

Appendix: Decision Log

Every decision taken in the five notebooks, why it was taken, and what the saved results show happened. Notebook 01 Rishik, 02 Praneeth, 03 Srikar, 04 Sami, 05 Mani Kanta.

#	NB	Decision	Why / alternative	What happened	Verdict
1	01	Binary classification: delayed = arrives ≥15 min late	Decision is yes/no; BTS definition; delay minutes skewed (skew 3.36). Regression on minutes rejected.	327,346 labelled flights, 0.245 delayed.	adopted
2	01	Predict at 05:00 on day D; post-departure columns banned; history ends on D−1	Early enough to act; about 3% of the previous day's flights land after midnight, so "end of previous day" is not exact.	Leakage tests pass (2 passed in 7.08s); no post-departure column is a feature.	worked
3	01	Cancelled/diverted flights unlabelled but kept on the schedule	Label undefined; inventing one would fabricate data. They still occupy the schedule.	8,255 cancelled + 1,175 diverted excluded from training and scoring, counted in congestion features.	adopted
4	01	Metric fixed before modelling: PR AUC (+ ROC AUC, Brier, ECE, lift)	Accuracy rewards "always on time".	"Always on time" scores 0.863 accuracy in September with zero recall; random scores PR AUC ≈ base rate.	worked
5	01	Chronological split: warm-up Jan 1–14, train to Aug, valid Sep, calib Oct, test Nov–Dec (once)	Same-day flights share weather; a random split leaks it.	Shares 65.2/8.6/9.1/17.1%. Monthly delay rate ranges 0.137 (Sep) to 0.336 (Dec): a base-rate shift that later drives the calibration and threshold results.	worked
6	01	Confidence intervals by day-block bootstrap	Flights on one day are not independent.	Test CIs are about ±0.05 wide (LR [0.351, 0.459]), which is what makes the LR/boosting gap non-significant.	worked
7	02	EDA and every preprocessing statistic on training rows only	Looking at valid/test while deciding is peeking.	Pipelines fitted on train; imputer median of plane age (11.0) is the training median.	adopted
8	02	Impute aircraft fields (median + missing indicator); do not delete rows	17.47% missing, concentrated in MQ (95.1%) and AA (69.5%): systematic. Deleting would bias two airlines.	Indicator columns keep the "unknown aircraft" signal; no rows lost.	adopted
9	02	Keep delay outliers	9.1% of flights are IQR outliers, but they are real delays and the yes/no label ignores size.	No rows removed.	adopted
10	02	Check duplicates	Worksheet checklist item 4.	0 full and 0 key duplicates: nothing to remove.	no action needed
11	02	History windows chosen from ACF/PACF of the daily delay rate	ACF lag 1 = 0.343, PACF lag 2 = -0.109, ACF lag 7 = 0.008 ⇒ 1-day window + day-of-week features; 7/28-day windows only for small groups.	In notebook 03 the history group raised September PR AUC from 0.209 to 0.261.	worked
12	02	No SMOTE / re-sampling	Imbalance is moderate (0.268 delayed); re-sampling distorts probabilities.	LR's raw test probabilities were already well calibrated (ECE 0.029).	worked
13	02	Standard (z-score) scaler for the linear model	Compared with a robust scaler.	September PR AUC 0.1974 vs 0.1978: a tie, standard kept.	tie
14	02	One-hot for linear models, ordinal codes for trees, native categoricals in XGBoost	Integer codes imply a fake order to a linear model; trees only ask "code ≤ k?".	LR validation matrix: 120 columns.	adopted
15	03	Five feature groups, 41 numeric + 3 categorical	Each group motivated by an EDA finding.	LR with all five groups is best on both months (Sep 0.261, Oct 0.306). History adds +0.052 in Sep but only +0.002 in Oct.	worked
16	03	Shrink history rates towards the overall rate ($\alpha = 10$)	A flight number seen 3 times gives an extreme, noisy rate.	Small groups pulled to the mean; log-volume features say how much evidence a rate rests on.	adopted
17	03	No month or day-of-year feature	Test months never appear in training, so month would act as a date ID.	Seasonality enters only through history features (see drift, notebook 05).	adopted
18	03	Keep all features unless a method beats them on both September and October	Filter (MI), VIF pruning, SFS (from scratch), L1, PCA tried.	None passed. SFS: Sep 0.279 > 0.261 but Oct 0.281 < 0.306: it over-fitted the month it was selected on.	all features kept
19	03	Report VIF but keep collinear features	13 features have VIF > 10; collinearity hurts interpretation, not prediction.	Pruning 7 features lowered both months (0.256 / 0.302).	worked
20	04	Baselines before any learned model	Phase 7: a complex model must beat a simple rule.	Sep / Oct PR AUC: majority 0.137 / 0.176; flight's 28-day rate 0.190 / 0.269; scheduled hour 0.188 / 0.229.	floor set

#	NB	Decision	Why / alternative	What happened	Verdict
21	04	Logistic regression from scratch (batch, mini-batch, SGD), checked against scikit-learn	Course training loop; log loss = Bernoulli MLE.	Batch GD train log loss 0.5279 vs scikit-learn 0.5278; mini-batch 0.5282 in 5 epochs.	verified
22	04	Learning-rate sweep for batch GD	Show slow vs divergent steps.	lr 0.01 → 0.6195 after 100 epochs (slow); 0.5 → 0.5282 (best); 8 → 2.3942 (diverged).	as expected
23	04	Choose L2 strength C on September only	October and test must not choose settings.	C = 0.001 (Sep 0.2655); scores barely move with C, so LR is bias-limited, not variance-limited.	adopted
24	04	Decision-tree depth sweep	Show over-fitting.	Unlimited depth: train 1.000, Sep 0.140; best Sep at depth 2 (0.180).	as expected
25	04	Compare families on rolling Jul/Aug/Sep with fixed configurations	September alone is unusually calm; fixed settings avoid using future months.	Mean PR AUC: LR 0.408 > random forest 0.386 > untuned XGBoost 0.366 > tree 0.328. LR is the model to beat; boosting goes on to tuning.	LR leads
26	05	Gradient boosting from scratch, checked against scikit-learn	Prove the theory (trees fitted to $r = y - p$, Newton leaf values).	September log loss 0.4165 and PR AUC 0.208, identical to scikit-learn.	verified
27	05	Honest early stopping: stop on month $m-1$, refit, score m	Early-stopping on the scored month lets the stopping rule see the answers.	Same configuration: optimistic mean 0.402 vs honest 0.382 (+0.020 inflation avoided).	trap found
28	05	Random search, 8 of 288 XGBoost configurations, honest rolling score	Tuned boosting is the fair test of the project's model.	Best: lr 0.03, depth 3, min child weight 50, subsample 0.7, colsample 0.5, λ 1.0 → 0.386. All 8 lie in 0.376–0.386, below LR (0.409).	boosting behind LR
29	05	Hybrid: XGBoost starts from LR's logit (base_margin) and learns corrections	Zero trees = plain LR, so it should never be much worse.	Rolling mean 0.406 (LR 0.409, boosting 0.386); trees per fold [16, 1219, 15].	matches LR
30	05	Final tree count = median of the fold counts	One calm month would stop far too early.	Boosting folds [40, 110, 13] → 40 trees; hybrid → 16; September alone would pick 1.	worked
31	05	Refit on Jan 15–Sep 30; Platt-calibrate on October	Use the most recent month; calibrate on a month no model has seen.	Boosting ECE 0.034 → 0.023 (better); LR 0.029 → 0.035 and hybrid 0.028 → 0.032 (worse: already calibrated, then October's lower base rate pulled them).	mixed
32	05	Test set (Nov–Dec) scored once, all models together	Golden barrier.	PR AUC: hybrid 0.403, LR 0.403, boosting 0.390, rule 0.369, majority 0.258. Intervals overlap: no winner among learned models.	no winner
33	05	Flag policy: calibrated $p \geq 0.5$ instead of the October score threshold	Delay rate rose from 0.176 (Oct) to 0.258 (test).	October threshold: precision 0.538–0.547, recall 0.053–0.056, 25–26 flagged per 1000. Calibrated rule: precision 0.517–0.529, recall 0.085–0.097, 42–49 flagged, about 2× a random list.	rule (b) chosen
34	05	Importance and ablation on October, never on test	Analysis must not shape the reported score.	Permutation: sched_hour 0.01726, h_flight_28d_rate 0.01424. Ablation: peak 0.303 with schedule+calendar+aircraft; + congestion 0.291; + history 0.264.	drift found
35	05	Error analysis by month and airport (lift)	Where does each model fail?	Boosting is weakest in November (lift 1.38 vs LR 1.58) and at JFK (1.44 vs 1.54); December is hard for all (about 1.35).	boosting weaker in Nov